Agenda

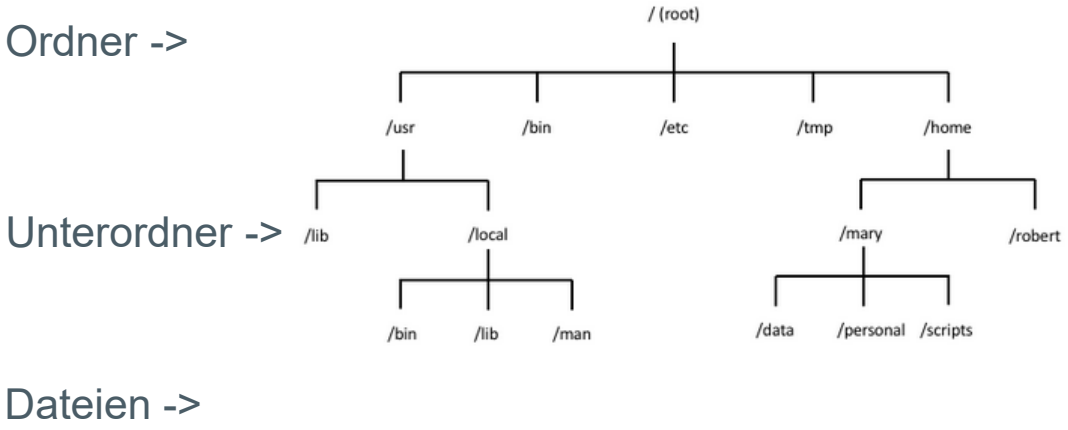
1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Dateisysteme

- Eine **Datei** ist eine beliebig lange Folge von Bytes, welche inhaltlich zusammengehören.

- Ein **Dateisystem** ist eine Menge von Datenstrukturen und Systemaufrufen, die von einem Betriebssystem benutzt werden, um Dateien zu speichern.
- Ein Dateisystem abstrahiert von der darunterliegenden Hardware.
  Zum Beispiel sehen Sie im Windows-Explorer nicht nur Ihre Festplatte(n), sondern auch USB-Sticks, etc.
- Dateisysteme sind **flach** oder **hierarchisch** (siehe Bild) organisiert.





UNIX unterscheidet verschiedene Arten von Dateien

- **Reguläre Dateien**
  Dateien enthalten Daten. UNIX unterscheidet das Format der Daten nicht (wohl aber die Anwendungen, die mit den Dateien arbeiten.

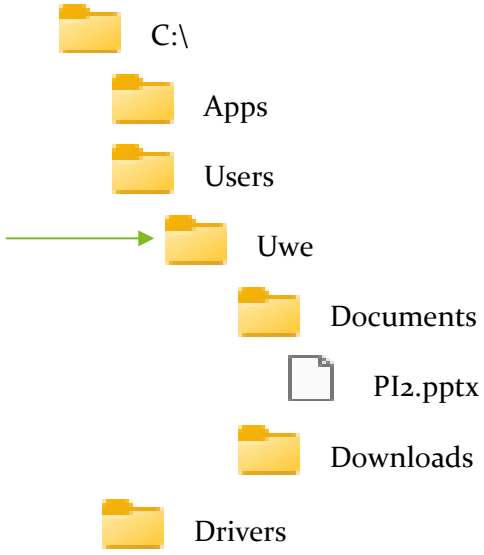
- **Spezielle Dateien**
  Jedes Gerät ist verbunden mit einer speziellen Datei (zum Beispiel `/dev/console`). Um auf ein Gerät zuzugreifen, wird die verbundene Datei geöffnet. Zum Empfangen von Daten oder Senden von Daten wird von der Datei gelesen bzw. auf ihr geschrieben.

- **Links** (Verweise)
  - Harte Links
    Ein harter Link ist ein Verweis auf eine Datei im gleichen Dateisystem
  - Symbolischer Link
    … auf eine Datei in einem anderen Dateisystem

- **Verzeichnisse** („Directories“)
  - Ein Verzeichnis enthält eine Menge von Dateien und Verzeichnissen.
  - Das oberste Verzeichnis ist `/`
  - Programme können Verzeichnisse öffnen und lesen, schreiben darf aber nur das Betriebssystem selbst.
  - Jede Datei wird durch einen Pfadnamen identifiziert (z.B. `/home/uwe/THM/pi2/slides.pptx`).
  - Jedes Verzeichnis hat eine Datei `..`, die auf das darüberliegende Verzeichnis zeigt.
  - `.` zeigt auf das Verzeichnis selbst.

Absolute und relative Pfade – Beispiel Windows

Aktuelles working directory



	Relative Pfade	Absolute Pfade
C:\	..\..\..	C:\
Apps	..\..\Apps	C:\Apps
Users	..	C:\ Users
Uwe	.	C:\ Users \Uwe
Documents	Documents	C:\ Users \Uwe\Documents
PI2.pptx	Documents\PI2.pptx	C:\ Users \Uwe\Documents\PI2.pptx
Downloads	Downloads	C:\ Users \Uwe\Downloads
Drivers	..\..\Drivers	C:\Drivers

Wichtige Shell-Kommandos zum Arbeiten mit Dateien

- `pwd` (print working directory)
  Gibt den Namen des aktuellen Verzeichnisses aus
- `ls` (list)
  Gibt die Liste der Dateien im aktuellen Verzeichnis bzw. des Argumentes aus
  Bsp. `ls /home`
- `cd` (change directory)
  Wechselt das Verzeichnis
- `cp` (copy), `mv` (move), `rm` (remove)
  Kopiert, verschiebt oder löscht eine oder mehrere Dateien
- `mkdir, rmdir`
  `mkdir` Legt ein neues Verzeichnis an, `rmdir` löscht ein Verzeichnis
- `cat, more` und `less`
  Anzeigen des Inhaltes einer Datei

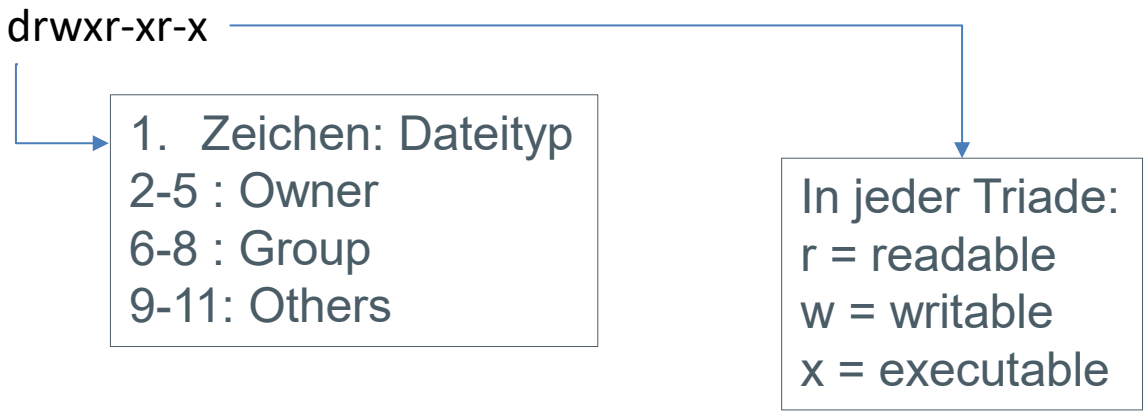
Inodes (Index Nodes)

- Linux verwendet sogenannte Blöcke (auch manchmal **Cluster** genannt), um Dateien zu speichern.
- Die Größe eines Blocks wird bei der Formatierung festgelegt und beträgt zwischen 4 und 32 KB.
  - Große Blöcke: Für kleine Dateien wird viel Platz verschwendet.
  - Kleine Blöcke: Mehr Verwaltungsaufwand für große Dateien.
- Immer, wenn eine Datei (auch ein Verzeichnis!) angelegt wird, wird auch ein **inode** angelegt. Dieser enthält unter anderem Informationen über:
  - Besitzer der Datei
  - Zugriffsrechte
  - Datum
  - Größe
  - Liste von 15 Clustern (wobei die letzten 3 auf Cluster von Clustern …) verweisen.

Zugriffsrechte (Permissions) in Unix

```
uwe@DESKTOP-0O7TS1G:~$ ls -l
total 84
-rwxr-xr-x 1 uwe uwe 14472 Feb 16 08:13 a.out
-rw-r--r-- 1 uwe uwe    61 Feb 16 08:13 hello.c
-rw-r--r-- 1 uwe uwe  1504 Feb 16 08:14 hello.o
-rw-r--r-- 1 uwe uwe 56786 Feb 16 12:09 isa.txt
drwxr-xr-x 8 uwe uwe  4096 Nov 16 08:02 jdk-21.0.1
lrwxrwxrwx 1 uwe uwe    16 Oct 19 17:11 windows -> /mnt/c/Users/uwe
```

d=directory
l=link



Beispiel:
Die Datei a.out darf jeder lesen und ausführen, nur der Besitzer darf sie schreiben oder löschen.

Numerische Notation

Jeder Triade kann man eine Zahl von 0 bis 7 zuordnen, indem man

- Für ein gesetztes r bit die Zahl 4 nimmt
- Für ein gesetztes w bit 2 addiert
- Für ein gesetztes x bit 1 addiert.

Beispiele:

```
rwxr-xr--     754
rwxrwxrwx     777
rwx------     700
```

Mit dem Befehl `chmod` kann man die Permissions anpassen:

```
$ ls -l isa.txt
-rw-r--r-- 1 uwe uwe 56786 Feb 16 12:09 isa.txt
$ chmod g+w isa.txt
$ ls -l isa.txt
-rw-rw-r-- 1 uwe uwe 56786 Feb 16 12:09 isa.txt
$ chmod 644 isa.txt
$ ls –l isa.txt
-rw-r--r-- 1 uwe uwe 56786 Feb 16 12:09 isa.txt
```

Agenda

1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	**Prozesse**	**77**
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Motivation

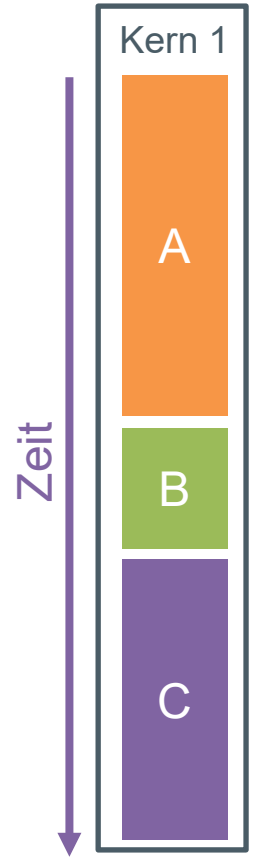
Häufig möchten wir mehrere Anwendungen gleichzeitig nutzen:

- Mehrere Webserver auf einem Computer
- Benutzer möchten mehrere Aufgaben gleichzeitig erledigen (z.B. Musikplayer, PDF-Reader, Editor und Spiele-Download)

Diese Aufgaben sind nicht schnell “ausgerechnet”, sondern laufen über einen längeren Zeitraum → Sequenzielle Verarbeitung ist **nicht möglich**, da z.B. ein 15 Minuten Video dann 15 Minuten die CPU blockiert.

Heutige Computersysteme sind auf Multi-Tasking ausgelegt:

1. Systeme enthalten mehrere CPU-Kerne für parallele Berechnungen
2. Betriebssysteme stellen Mechanismen bereit, um mehrere Prozesse auf dem gleichen CPU-Kern “quasi-parallel” auszuführen



Sequentielles Abarbeiten von Prozessen

z.B.
erst Prozess A,
dann Prozess B,
dann Prozess C

B startet erst, wenn A komplett fertig und beendet wurde
…

Nebenläufigkeit und Parallelität

Nebenläufigkeit
Aktivitäten haben keine kausale Abhängigkeit voneinander
→ Resultate von A werden nicht für B benötigt (und umgekehrt)

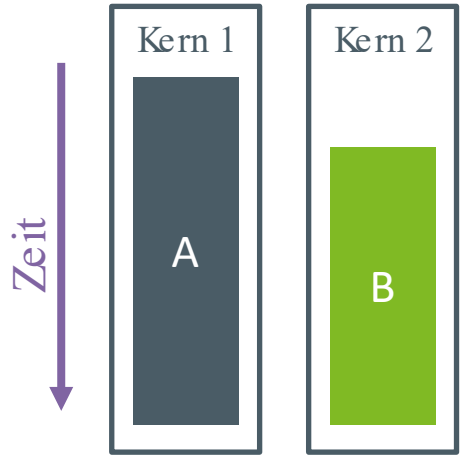
Zeitliche Verzahnung
Wechsel zwischen Aktivitäten
auf dem gleichen CPU Kern



Der CPU-Kern wechselt zwischen Teilstücken der Aktivitäten A und B.

Das Betriebssystem bestimmt die Länge und Reihenfolge.

Parallelität
Aktivitäten laufen gleichzeitig
auf unterschiedlichen CPU Kernen

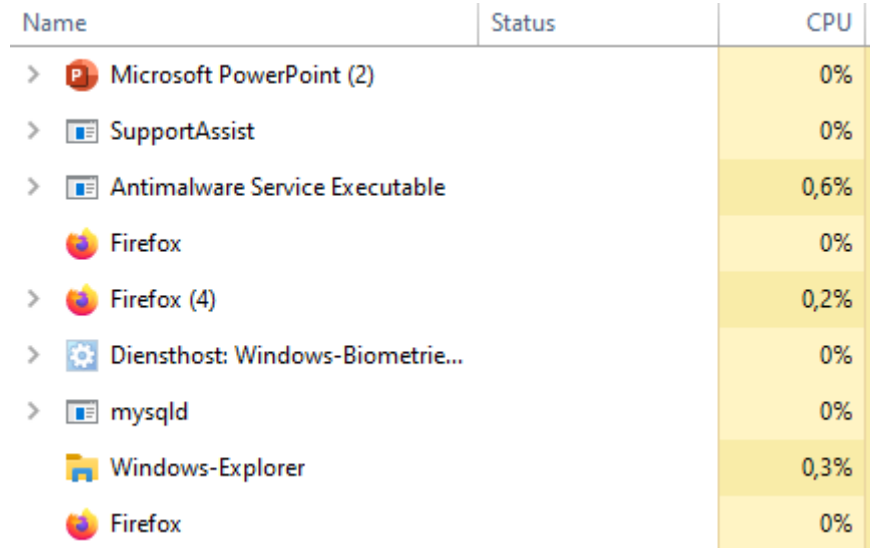


Aktivitäten A und B laufen zeitgleich (=parallel) auf verschiedenen CPU-Kernen

Was ist ein Prozess?

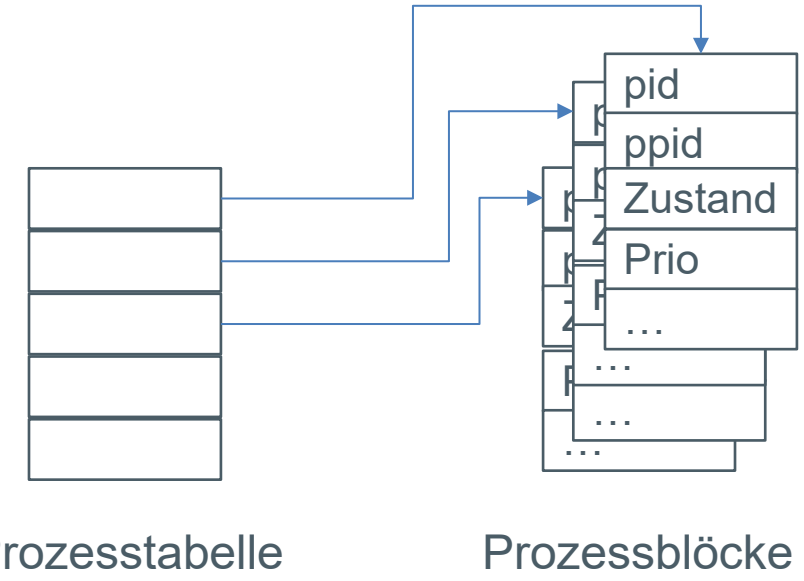
- Lat. *Procedere* : voranschreiten
- Ein Prozess ist die Instanz eines Programms in Ausführung.
- Ein Prozess beinhaltet außerdem den Kontext des Programms:
  - Hardware-Kontext: Aktuelle Werte der Register, PC, …
  - Benutzer-Kontext: Daten im Speicher des Prozesses
  - Systemkontext: Daten, die das Betriebssystem über den Prozess hält, wie z.B. Prozessnummer, Zustand, Priorität, …





Prozesskontrollblock (UNIX: PCB, Windows: PEB (process environment block))

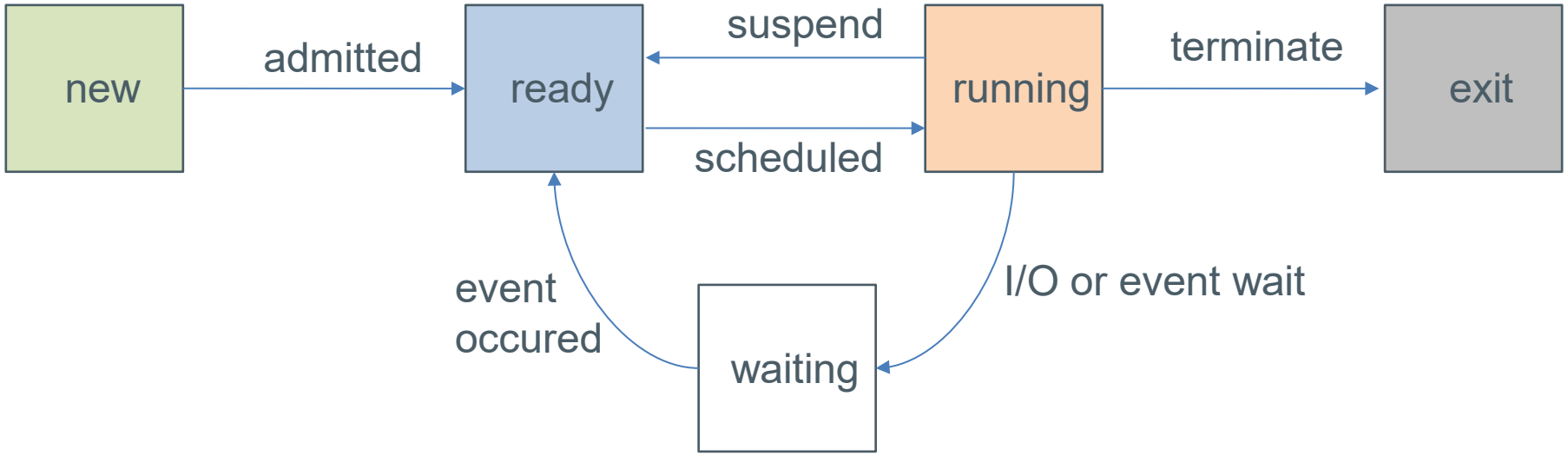
- In der **Prozesstabelle** existiert zu jedem Prozess ein Prozesskontrollblock.
- Im Prozesskontrollblock verwaltet das Betriebssystem:
  - Prozess-ID (PID)
  - ID des Vater-Prozesses (PPID)
  - Zustand
  - Priorität
  - …



Prozesszustände

Welche Informationen muss der **Zustand** enthalten?

- Prozess wird gerade ausgeführt (running)
- Beendet (exit)
- Neu (new)
- Ist bereit ausgeführt zu werden (ready)
- Ist blockiert, indem er z.B. auf einen Dateizugriff wartet (waiting)



Wie erzeugt man einen Prozess?

Forking:

- Ein Vater-Prozess erzeugt eine identische Kopie seiner selbst als Kindprozess mittels `fork()`.

Chaining:

- Ein Prozess startet mit `exec()` einen neuen Prozess und wird damit selber beendet.

Frage: Wie startet die Shell ein Programm?

Antwort: Die Shell kopiert sich selbst mittels `fork()` und die Kopie startet mit `exec()` das Programm. (**Creation**)

Das ps Kommando

Das Kommando `ps` zeigt Informationen zu Prozessen an:

Alle Prozesse des Benutzers

```
$ ps
  PID TTY          TIME CMD
   15 pts/0    00:00:00 bash
   73 pts/0    00:00:00 ps
```

Alle Prozesse aller Benutzer

```
$ ps ax
  PID TTY      STAT   TIME COMMAND
    1 ?        Sl     0:00 /init
   13 ?        Ss     0:00 /init
   14 ?        S      0:00 /init
   15 pts/0    Ss     0:00 -bash
   89 pts/0    R+     0:00 ps ax
```

Vordergrund und Hintergrund

- Startet die Shell ein Programm, läuft es im Vordergrund, d.h. das Programm „besitzt“ das Terminal, d.h., es nimmt Eingaben entgegen und macht Ausgaben.
- Die Shell läuft während dieser Zeit im Hintergrund.
- Wird das Programm beendet, kommt die Shell wieder in den Vordergrund.

- Programme können auch direkt im Hintergrund gestartet werden:

  z.B: `$ javac *.java &`

  Startet den Java-Compiler im Hintergrund.

- Wichtige Kommandos:

  `jobs` : Gibt Informationen über die im Hintergrund laufenden Prozesse aus

  `fg [job_spec]` : Holt den angegebenen Prozess (Job) in den Vordergrund

  `bg [job_spec]` : Schiebt den angegebenen Prozess in den Hintergrund